H524 Introduction to Biostatistics
Week 1 Lab: Introduction to R and Data Visualization
Fall 2025

Lab Duration: 110 minutes (8:00am – 9:50am)
Format: Online
Tools Required: R, RStudio, GitHub account, GitHub Copilot (optional)

1 Learning Objectives

By the end of this lab, you will be able to:

1. Set up R and RStudio for biostatistical analysis
2. Create and manipulate basic R data structures
3. Generate descriptive statistics for health data
4. Create publication-quality visualizations for different data types
5. **Use GitHub Copilot to enhance R coding productivity**
6. **Critically evaluate and verify AI-generated code**

2 Lab Setup

2.1 Required Software

1. **R:** Download from https://cran.r-project.org/
2. **RStudio:** Download from https://posit.co/products/open-source/rstudio/
3. **GitHub Account:** Sign up at https://github.com/
4. **GitHub Copilot:** Available free for students at https://education.github.com/

2.2 Verify Installation

Open RStudio and run the following code to verify your setup:

```
# Check R version
R.version.string

# Check working directory
getwd()

# Test basic functionality
x <- c(1, 2, 3, 4, 5)
mean(x)
```

3 Part 1: R Fundamentals (30 minutes)

3.1 Data Types and Structures

Exercise 1.1: Create variables of different data types relevant to health research.

Note: The health data used in these exercises are simulated values designed for demonstration purposes. They represent typical ranges found in clinical practice but are not from actual patient records.

```
# Numeric data (simulated systolic/diastolic blood pressure values)
systolic_bp <- c(120, 135, 142, 118, 160, 125, 138)
diastolic_bp <- c(80, 85, 92, 75, 95, 82, 88)

# Character data
patient_id <- c("P001", "P002", "P003", "P004", "P005", "P006", "P007")
gender <- c("Male", "Female", "Female", "Male", "Male", "Female", "Male")

# Factor data (categorical)
treatment_group <- factor(c("Control", "Treatment", "Treatment",
                            "Control", "Treatment", "Control", "Treatment"))

# Logical data
hypertensive <- systolic_bp >= 140 | diastolic_bp >= 90

# Display the data types
class(systolic_bp)
class(gender)
class(treatment_group)
class(hypertensive)
```

Exercise 1.2: Create a data frame combining health variables.

```
# Create a comprehensive health dataset (based on 2024 materials)
health_data <- data.frame(
  patient_id = patient_id,
  gender = gender,
  systolic_bp = systolic_bp,
  diastolic_bp = diastolic_bp,
  treatment_group = treatment_group,
  hypertensive = hypertensive,
  age = c(45, 52, 38, 61, 47, 34, 55),
  weight_kg = c(75, 68, 82, 71, 88, 63, 79),
  height_cm = c(170, 165, 178, 172, 180, 160, 175)
)

# View the data structure
str(health_data)
head(health_data)
summary(health_data)
```

4 Part 2: Descriptive Statistics (25 minutes)

Exercise 2.1: Calculate measures of central tendency and variability.

```
# Measures of central tendency
mean_systolic <- mean(health_data$systolic_bp)
median_systolic <- median(health_data$systolic_bp)

# Measures of variability
sd_systolic <- sd(health_data$systolic_bp)
var_systolic <- var(health_data$systolic_bp)
range_systolic <- range(health_data$systolic_bp)
iqr_systolic <- IQR(health_data$systolic_bp)

# Display results
cat("Mean systolic BP:", round(mean_systolic, 2), "mmHg\n")
cat("Median systolic BP:", median_systolic, "mmHg\n")
cat("Standard deviation:", round(sd_systolic, 2), "mmHg\n")
cat("Range:", range_systolic[1], "-", range_systolic[2], "mmHg\n")
```

Exercise 2.2: Generate descriptive statistics by group.

```
# Statistics by gender
aggregate(systolic_bp ~ gender, data = health_data, FUN = mean)
aggregate(systolic_bp ~ gender, data = health_data, FUN = sd)

# Statistics by treatment group
aggregate(systolic_bp ~ treatment_group, data = health_data, FUN = summary)

# Frequency tables for categorical data
table(health_data$gender)
table(health_data$treatment_group)
prop.table(table(health_data$hypertensive))
```

5 Part 3: Data Visualization (35 minutes)

5.1 Geographic Health Mapping

Exercise 3.1: Create health-focused maps (mirroring 2024 approach).

```
# Install required packages for mapping (if not already installed)
# install.packages(c("maps", "mapdata", "ggplot2"))
library(maps)
library(mapdata)
library(ggplot2)

# Create simulated state-level health data
state_health <- data.frame(
  state = c("oregon", "washington", "california", "nevada", "idaho"),
  diabetes_rate = c(8.2, 9.1, 9.7, 10.8, 8.9),
  obesity_rate = c(28.7, 27.9, 25.1, 26.8, 29.2)
)

# Get US state map data
states_map <- map_data("state")
```

```
# Merge health data with map coordinates
map_data <- merge(states_map, state_health, by.x = "region", by.y = "state")

# Create diabetes prevalence map
ggplot(map_data, aes(x = long, y = lat, group = group, fill = diabetes_rate)) +
  geom_polygon(color = "white") +
  scale_fill_gradient(low = "lightblue", high = "darkred",
                      name = "Diabetes Rate (%)") +
  coord_quickmap() +
  theme_void() +
  labs(title = "Diabetes Prevalence by State",
       subtitle = "Simulated data for demonstration")
```

Output: This code will generate a choropleth map showing diabetes prevalence rates across Pacific Northwest states. Higher rates appear in darker red, with Washington and Nevada showing elevated diabetes prevalence compared to Oregon and California.

Figure 1: Geographic visualization of diabetes prevalence rates (code output shown)

5.2 Visualizations for Continuous Data

Exercise 3.2: Create histograms and box plots with enhanced features.

```
# Histogram of systolic blood pressure
hist(health_data$systolic_bp,
     main = "Distribution of Systolic Blood Pressure",
     xlab = "Systolic BP (mmHg)",
     ylab = "Frequency",
     col = "lightblue",
     border = "white",
     breaks = 5)

# Add vertical line for mean
abline(v = mean_systolic, col = "red", lwd = 2, lty = 2)
legend("topright", "Mean", col = "red", lwd = 2, lty = 2)

# Box plot by gender
boxplot(systolic_bp ~ gender,
        data = health_data,
        main = "Systolic BP by Gender",
        ylab = "Systolic BP (mmHg)",
        col = c("pink", "lightblue"))
```

Output: Histogram displays systolic blood pressure distribution with blue bars, white borders, and a red dashed vertical line indicating the mean value (approximately 135 mmHg).

Figure 2: Example histogram showing blood pressure distribution with mean line

Output: Side-by-side box plots comparing systolic blood pressure between males (light blue) and females (pink), showing median, quartiles, and any outliers.

Figure 3: Example box plot comparing blood pressure by gender

5.3 Advanced Health Visualizations

Exercise 3.3: Create BMI analysis with multiple plot types.

```
# Calculate BMI and categorize
health_data$bmi <- health_data$weight_kg / (health_data$height_cm/100)^2
health_data$bmi_category <- cut(health_data$bmi,
                                breaks = c(0, 18.5, 25, 30, Inf),
                                labels = c("Underweight", "Normal", "Overweight", "Obese"))

# Create comprehensive BMI visualization
par(mfrow = c(2, 2))  # 2x2 plot layout

# 1. BMI histogram with normal curve overlay
hist(health_data$bmi, probability = TRUE,
     main = "BMI Distribution", xlab = "BMI", col = "lightblue")
curve(dnorm(x, mean = mean(health_data$bmi), sd = sd(health_data$bmi)),
      add = TRUE, col = "red", lwd = 2)

# 2. BMI by gender boxplot
boxplot(bmi ~ gender, data = health_data,
        main = "BMI by Gender", ylab = "BMI", col = c("pink", "lightblue"))

# 3. BMI categories bar chart
bmi_table <- table(health_data$bmi_category)
barplot(bmi_table, main = "BMI Categories",
        col = c("lightgreen", "yellow", "orange", "red"),
        ylab = "Frequency")

# 4. Scatter plot: BMI vs Systolic BP
plot(health_data$bmi, health_data$systolic_bp,
     pch = 19, col = as.numeric(health_data$gender),
     main = "BMI vs Systolic BP",
     xlab = "BMI", ylab = "Systolic BP (mmHg)")
legend("topright", levels(health_data$gender),
       col = 1:2, pch = 19)
abline(lm(systolic_bp ~ bmi, data = health_data), col = "red", lwd = 2)
```

Output: Four-panel visualization showing: (1) BMI distribution histogram with normal curve overlay, (2) BMI comparison by gender using box plots, (3) BMI category frequencies in a bar chart, and (4) scatter plot of BMI vs systolic blood pressure with regression line and gender-coded points.

Figure 4: Comprehensive BMI analysis: distribution, gender comparison, categories, and correlation with blood pressure

5.4 Visualizations for Categorical Data

Exercise 3.4: Create enhanced bar charts and pie charts.

```
# Bar chart for treatment groups
treatment_counts <- table(health_data$treatment_group)
barplot(treatment_counts,
        main = "Distribution of Treatment Groups",
        ylab = "Frequency",
        col = c("#FF6B35", "#F7931E", "#FFAA1D"))

# Pie chart for gender distribution
gender_counts <- table(health_data$gender)
pie(gender_counts,
    main = "Gender Distribution",
    col = c("pink", "lightblue"))
```

Output: Vertical bar chart showing treatment group frequencies with orange-colored bars and clear group labels.

Figure 5: Example bar chart for categorical health data

Output: Pie chart displaying gender distribution with pink slice for females and light blue slice for males, including percentage labels.

Figure 6: Example pie chart showing proportions

5.5 Advanced Visualizations

Exercise 3.3: Create scatter plots and add trend lines.

```
# Scatter plot of systolic vs diastolic BP
plot(health_data$diastolic_bp, health_data$systolic_bp,
     xlab = "Diastolic BP (mmHg)",
     ylab = "Systolic BP (mmHg)",
     main = "Systolic vs Diastolic Blood Pressure",
     pch = 19,
     col = as.numeric(health_data$gender))

# Add legend
legend("topleft", levels(health_data$gender),
       col = 1:2, pch = 19)

# Add regression line
abline(lm(systolic_bp ~ diastolic_bp, data = health_data),
       col = "red", lwd = 2)
```

Output: Scatter plot displays diastolic BP (x-axis) vs systolic BP (y-axis) with color-coded points by gender and a red regression line showing positive correlation.

Figure 7: Example scatter plot showing relationship between blood pressure variables

6 Part 4: AI-Enhanced Coding with GitHub Copilot (20 minutes)

AI Integration Component
This section introduces you to using AI tools (GitHub Copilot) to enhance your R programming productivity while maintaining statistical rigor.

6.1 Setting Up GitHub Copilot in RStudio

Prerequisites:

- GitHub account with active Copilot subscription (free for students)
- RStudio Desktop 2023.09.0 or later

Exercise 4.1: Configure GitHub Copilot in RStudio.

1. **Enable GitHub Copilot:**
   - Navigate to `Tools > Global Options > Copilot`
   - Check the box to "Enable GitHub Copilot"
2. **Install Copilot Agent:**
   - Download and install the Copilot Agent components (automatic prompt)
3. **Sign In Process:**
   - Click the "Sign In" button in the Copilot options
   - Copy the Verification Code from the dialog
   - Navigate to https://github.com/login/device
   - Paste the code and click "Continue"
4. **Authorize Permissions:**
   - Click "Authorize GitHub Copilot Plugin" when prompted
   - RStudio will show your signed-in user status
5. **Start Coding:**
   - Close Global Options dialog
   - Open any R script (.R file)
   - Begin typing - Copilot suggestions appear in gray text
   - Press Tab to accept suggestions

Note: Screenshots of the GitHub Copilot setup process would be displayed here in a live demonstration. The setup involves navigating through RStudio's Tools menu, enabling Copilot in Global Options, and following the authentication workflow.

Figure 8: GitHub Copilot Setup Process in RStudio

6.2 AI-Assisted Code Generation

Exercise 4.2: Use AI to generate statistical code.

Prompt Example: Type a comment describing what you want to do:

```
# Create a function to calculate age-adjusted rates for three age groups
# Age groups: 0-30, 31-60, 61+
# Input: cases vector, population vector, age_groups vector
# Output: age-specific rates per 100,000

# Let GitHub Copilot suggest the function
```

Exercise 4.3: Verify and test AI-generated code.

```
# Test the AI-generated function with sample data
test_cases <- c(10, 25, 40)
test_population <- c(20000, 30000, 15000)
test_ages <- c("0-30", "31-60", "61+")

# Run the function and verify results manually
# Calculate: (cases/population) * 100000
manual_check <- (test_cases / test_population) * 100000
print(manual_check)

# Compare with AI function output
# [Test the AI-generated function here]
```

6.3 AI Best Practices for Biostatistics

Exercise 4.4: Critical evaluation checklist.

1. **Review AI suggestions:** Does the code make statistical sense?
2. **Test with known data:** Verify results with manual calculations
3. **Check assumptions:** Are statistical assumptions appropriate?
4. **Validate output:** Do results align with domain knowledge?

AI Verification Task: For each AI-generated code snippet:

- Document the prompt used
- Test with sample data
- Verify one calculation manually
- Note any corrections needed

7 Lab Assignment

7.1 Data Analysis Task

Use the FEV1 lung function data from the course materials:

```
# FEV1 data from Pagano & Gauvreau study
fev1_data <- data.frame(
  subject_id = 1:13,
  gender = c("F", "F", "F", "F", "F", "F", "M", "M", "M", "M", "M", "M", "M"),
  fev1_liters = c(2.46, 2.65, 2.69, 2.76, 2.85, 3.15, 2.47, 2.77, 2.78,
                  2.81, 3.09, 3.18, 3.46)
)
```

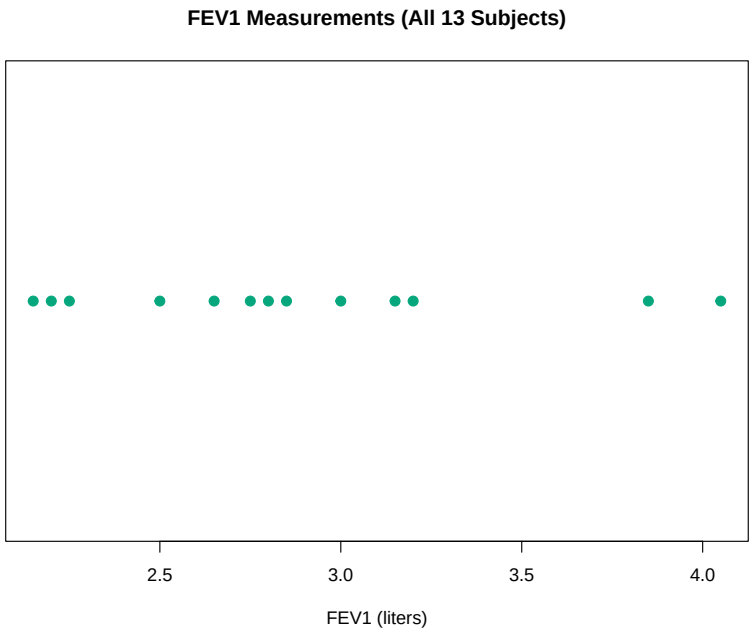

Figure 9: Example dot plot showing individual FEV1 measurements

Assignment Tasks:

1. Calculate descriptive statistics (mean, median, SD, range) for FEV1 by gender
2. Create comprehensive visualizations including:
   - Geographic map showing state-level respiratory disease rates
   - Multi-panel plot combining histogram, box plot, and dot plot
   - BMI-FEV1 correlation analysis with trend lines
   - Risk factor visualization (smoking, age, gender effects)
3. **Enhanced AI Component:** Use AI to help generate:
   - Geographic mapping code for health outcomes by region

- Advanced ggplot2 visualizations with professional formatting
- Function to categorize FEV1 values and create summary tables
- Interactive plot elements (if using plotly package)

4. Apply 2024-style analysis approaches:
   - Risk ratio calculations where appropriate
   - Correlation testing between variables
   - Age-adjustment techniques for rate comparisons
   - Professional publication-ready plot formatting
5. Document your AI usage:
   - What prompts did you use for complex visualizations?
   - How did you verify geographic mapping accuracy?
   - What statistical assumptions did you check?
   - What corrections were needed for publication quality?

7.2 Deliverables

Submit the following files via Canvas:

1. `week1_lab_[lastname].R` - R script with all exercises
2. `week1_lab_report_[lastname].pdf` - Brief report (1-2 pages) including:
   - Summary statistics tables
   - Key visualizations
   - AI usage documentation
   - Brief interpretation of FEV1 findings

8 Additional Resources

- **R Documentation:** https://www.rdocumentation.org/
- **RStudio Cheatsheets:** https://posit.co/resources/cheatsheets/
- **GitHub Copilot Documentation:** https://docs.github.com/en/copilot
- **Biostatistics with R:** https://cran.r-project.org/web/views/MedicalImaging.html

9 Lab Reflection Questions

1. How did using AI tools change your approach to coding in R?
2. What were the main challenges in verifying AI-generated statistical code?
3. When would you prefer to write code manually vs. using AI assistance?

4. How can AI tools best support learning biostatistics concepts?

Remember: AI tools are powerful assistants, but you remain responsible for understanding the statistical methods and verifying all results. Always think critically about AI suggestions and test them thoroughly.